

A Parallel Congruence Closure Algorithm

Zak Kent*, Amar Shah†

*Carnegie Mellon University, zkent@cmu.edu

†Carnegie Mellon University, amarshah@cmu.edu

Abstract—Abstract goes here.

I. INTRODUCTION

Satisfiability modulo theories (SMT) solvers are used to decide the satisfiability of first-order logic formulas. State-of-the-art solvers such as Z3 [1] and cvc5 [2] can reason about a wide variety of “theories” such as uninterpreted functions, linear arithmetic, bit-vectors, and many others.

SMT solvers are widely used in a variety of applications, including software verification, hardware design, and mathematical theorem proving. Hence, the performance of SMT solvers is critical, and much work has gone into improving their efficiency. One promising avenue for improving the performance of SMT solvers is *parallelization*: utilizing multiple processors to solve the problem faster.

SMT solvers, as the name suggests, consist of a Boolean satisfiability (SAT) solver to determine an assignment to the propositional skeleton and dedicated theory solvers to test the validity of theory lemmas. Existing work on parallel SMT, however, has focused on (a) portfolio approaches that run multiple differently-configured solver instances on the same input [3, 4] or (b) techniques that involve partitioning the search space and having solvers run on the partitioned search space [5–8]. In all of these, the underlying theory solvers still run single-threaded on each subproblem. There is limited work on parallelizing the theory solvers themselves. This is a significant gap, as theory solvers can be a bottleneck in SMT solving, and parallelizing them could lead to significant performance improvements.

To this end, we present a novel parallel congruence closure algorithm for the theory of uninterpreted functions. Our algorithm uses a concurrent union find data structure.

AS: clean up the algorithm description

At a high level, our algorithm replaces the classical sequential worklist of Nelson and Oppen [9] with a *bulk-synchronous* closure loop in the spirit of equality-saturation engines such as egg [10]. Initial equalities are unioned into a lock-free concurrent union-find [11], marking each affected e-class as *changed*. The algorithm then proceeds in rounds. In each round we (i) collect the current frontier of changed classes in parallel, (ii) gather every parent e-node whose canonical signature could have shifted, (iii) hash each parent’s signature under the current union-find roots, (iv) group nodes by signature using a parallel semisort [12], and (v) union together every group that witnesses a new congruence. Each step is a pure data-parallel primitive—`filter`, `flat_map`, `map`, `semisort`,

and a parallel union pass—built on PARLAYLIB’s scheduler and sequence types [13]. The algorithm terminates when no round discovers a new merge, exactly mirroring the fixed point of the sequential procedure. The hashcons and the e-graph’s child-to-parent index are maintained across rounds without per-element locks: writers only append, and concurrent reads see a consistent partition because the union-find linearizes through compare-and-swap.

While congruence closure is P-complete [14], we show that our algorithm can achieve linear speedups on a synthetic benchmark suite. Our algorithm efficiently utilizes multiple cores and can be integrated into existing SMT solvers. We build a prototype SMT solver that incorporates our parallel congruence closure algorithm and evaluate it on a suite of SMT benchmarks, demonstrating significant performance improvements over a version with an existing sequential congruence closure algorithm.

We also evaluate our algorithm on a suite of SMT benchmarks from the annual SMT competition [?] and demonstrate a speedup of TODO% over a sequential implementation.

AS: current implementation does not have backtracking -> need to implement backtracking and re-evaluate.

Finally, we show that this parallel congruence closure algorithm is useful beyond SMT solving: we evaluate our algorithm on a set of equality saturation benchmarks [?] from the Herbie project [?] and demonstrate significant performance improvements over a sequential implementation.

AS: I emailed Oliver for this benchmarks suite

II. BACKGROUND

III. ALGORITHM

This section describes our parallel congruence closure procedure in detail. The algorithm rests on three components: a lock-free concurrent union-find that allows arbitrarily many threads to issue FIND/UNION concurrently (Section III-A); a parallel semisort that lets each round group congruent e-node candidates by signature without a sequential hash-table bottleneck (Section III-B); and a bulk-synchronous, rounds-based outer loop that drives both to a congruence-closed fixed point (Section III-C). All three build on PARLAYLIB [13], a shared-memory work-stealing runtime that supplies the sequence types and parallel primitives (`parallel_for`, `filter`, `flat_map`, `group_by_key`) that we use throughout.

A. Concurrent Union-Find

We use the lock-free concurrent disjoint-set-union of Jayanti and Tarjan [11]. A union-find of capacity n is represented as

an array of n atomic 32-bit words. Each word encodes either a *rank* (when the node is a root, indicated by the high bit) or a *parent pointer* into the same array (when the node is not a root). Both FIND and UNION are non-blocking: they read the current state, decide what update they would like to make, and install it via a compare-and-swap (CAS); on CAS failure they retry from a freshly-read state.

FIND(u) recursively walks parent pointers to a root, and on its way back attempts to install path compression by CAS-ing the walked node’s parent slot directly to the root. Because the CAS only succeeds when the slot still points to the old parent, lost races are harmless: another thread has already installed an at-least-as-good shortcut. UNION(u, v) first calls FIND on both endpoints, then attempts to CAS the smaller-rank root’s slot from a rank value to a parent pointer to the other root, retrying from the top of the routine on failure. Equal ranks tie-break by node id and attempt a best-effort rank increment on the winning root with a second CAS.

This UF supports the throughput regime that our outer loop demands: each round of congruence closure issues thousands to millions of FIND and UNION calls in parallel, and every operation linearizes regardless of contention.

B. Parallel Semisort

Once an e-node’s children have been canonicalized to their current UF roots, two e-nodes with the same operator and identical canonicalized child lists are *congruent* and must be merged. Detecting congruences therefore reduces to grouping e-nodes by signature. A sequential implementation would use a hash table; doing this in parallel requires either fine-grained locking or a sorting-based alternative.

We use the parallel *semisort* of Gu, Shun, Sun, and Blelloch [12]. Semisort is a relaxation of sorting that suffices for grouping: rather than producing a totally-ordered output, it produces a permutation of the input in which all elements with equal keys are adjacent, with no ordering required between groups. For a sequence of n items with m distinct keys, semisort runs in $O(n)$ work and $O(\log n)$ span on a CRCW PRAM, and avoids the $O(n \log n)$ work of a comparison-based sort.

In each closure round, we hash every candidate e-node’s signature into a 64-bit value, mod a power-of-two number of buckets chosen proportional to the number of threads. We hand the (key, payload) sequence to ParlayLib’s semisort, then process each bucket in parallel: within a bucket, a local sort plus an adjacent-pair scan identifies pairs of candidates that share a true signature (modulo hash collisions, checked via a full equality test on the canonicalized child lists).

C. Rounds-Based Congruence Closure

The classical Nelson–Oppen procedure [9] maintains a sequential worklist of e-classes whose use-sets must be re-examined after a merge. We replace it with a bulk-synchronous round structure that exposes parallelism within each round while preserving exactly the same fixed point. Algorithm 1 sketches the loop; all numbered steps are data-parallel.

Algorithm 1 Parallel congruence closure (one round)

Input: e-graph G with concurrent UF U , parent-index $P[\cdot]$, per-class *changed* flags $C[\cdot]$, after a batch of initial unions has marked each affected class.

Output: A new round’s worth of merges; loop until no merges fire.

- 1) *Frontier*. In parallel over class ids, *filter* C to collect $F = \{c : C[c]\}$ and clear the corresponding flags.
 - 2) *Gather*. `flat_map` F through P to obtain the list of candidate parent e-nodes whose signature might have shifted.
 - 3) *Canonicalize*. `map` each candidate n to a triple (h, idx, r) where h is a 64-bit hash of $(op(n), U.FIND(c_1), \dots, U.FIND(c_k))$, idx is the candidate’s position in the e-node array, and $r = U.FIND(class(n))$.
 - 4) *Group*. Hash h into one of B buckets and apply the parallel semisort of Section III-B.
 - 5) *Detect merges*. In parallel over buckets, sort within each bucket by $(h, \text{lexicographic canonicalized children})$ and scan adjacent pairs: every pair with equal signature whose representatives differ contributes a (r_a, r_b) pair to a new *merge list*.
 - 6) *Apply*. In parallel over the merge list, call $U.UNION(r_a, r_b)$ and set $C[r_a] = C[r_b] = \text{true}$.
-

The round repeats while step 6 produces any merges. Termination is guaranteed because each merge strictly reduces the number of distinct UF roots, and the loop discovers merges only through congruence; once the e-graph is closed, no candidate’s signature has shifted and the merge list is empty.

Several invariants make the parallel steps safe. The e-node array is append-only after the build phase, so step 3 reads but never mutates it. The parent index P may be appended to during build but is never re-ordered or freed during closure. The *changed* flags are a collection of independent atomic booleans, so step 6’s writes do not race destructively. The only shared mutable state during a round is the union-find itself, and that state is correct under arbitrary concurrency by Jayanti–Tarjan’s analysis. Because every step is a pure ParlayLib primitive, the entire round inherits ParlayLib’s work-stealing scheduler and its associated complexity bounds.

IV. EVALUATION

REFERENCES

- [1] L. de Moura and N. Bjørner, “Z3: An efficient SMT solver,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2008, pp. 337–340.
- [2] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar, “cvc5: A versatile and industrial-strength SMT solver,” in *Tools and Algorithms for the*

- Construction and Analysis of Systems (TACAS)*. Springer, 2022, pp. 415–442.
- [3] C. M. Wintersteiger, Y. Hamadi, and L. de Moura, “A concurrent portfolio approach to SMT solving,” in *Computer Aided Verification (CAV)*. Springer, 2009, pp. 715–720.
 - [4] C. Barrett, P.-W. Chen, B. Cook, B. Dutertre, R. B. Jones, N. Le, A. Reynolds, K. Sheth, C. Stephens, and M. W. Whalen, “SMT-D: New strategies for portfolio-based SMT solving,” in *Formal Methods in Computer-Aided Design (FMCAD)*, 2024.
 - [5] M. J. H. Heule, O. Kullmann, S. Wieringa, and A. Biere, “Cube and conquer: Guiding CDCL SAT solvers by lookaheads,” in *Hardware and Software: Verification and Testing (HVC)*. Springer, 2011, pp. 50–65.
 - [6] A. Wilson, A. Nötzli, A. Reynolds, B. Cook, C. Tinelli, and C. Barrett, “Partitioning strategies for distributed SMT solving,” in *Formal Methods in Computer-Aided Design (FMCAD)*, 2023, pp. 199–208.
 - [7] T. Kolárik, A. E. J. Hyvärinen, S. Asadzadeh, and N. Sharygina, “Parallel SMT solving via iterative tree partitioning,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2026.
 - [8] X. Cheng, M. Zhou, X. Song, M. Gu, and J. Sun, “Parallelizing SMT solving: Lazy decomposition and conciliation,” *Artificial Intelligence*, vol. 257, pp. 127–157, 2018.
 - [9] G. Nelson and D. C. Oppen, “Fast decision procedures based on congruence closure,” *Journal of the ACM*, vol. 27, no. 2, pp. 356–364, 1980.
 - [10] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panthekha, “egg: Fast and extensible equality saturation,” in *Principles of Programming Languages (POPL)*, 2021.
 - [11] S. V. Jayanti and R. E. Tarjan, “A randomized concurrent algorithm for disjoint set union,” in *ACM Symposium on Principles of Distributed Computing (PODC)*. ACM, 2016, pp. 75–82.
 - [12] Y. Gu, J. Shun, Y. Sun, and G. E. Blelloch, “A top-down parallel semisort,” in *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. ACM, 2015, pp. 24–34.
 - [13] G. E. Blelloch, D. Anderson, and L. Dhulipala, “ParlayLib – a toolkit for parallel algorithms on shared-memory multicore machines,” in *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. ACM, 2020, pp. 507–509.
 - [14] P. C. Kanellakis and P. Z. Revesz, “On the relationship of congruence closure and unification,” *Journal of Symbolic Computation*, vol. 7, no. 3, pp. 427–444, 1989, unification: Part 1. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0747717189800185>